

```

    expect_lte(1, 1)
  })
Test passed

> test_that ("expect lte", {
  expect_lte(1, 2)
})
Test passed

> test_that ("expect lte", {
  expect_lte(2, 1)
})
— Failure (Line 2): expect lte —————

2 is not less than 1. Difference: 1
Error: Test failed

> test_that ("expect gt", {
  expect_gt(2, 1)
})
Test passed

> test_that ("expect gt", {
  expect_gt(1, 2)
})
— Failure (Line 2): expect gt —————

1 is not strictly more than 2. Difference: -1
Error: Test failed

> test_that ("expect gte", {
  expect_gte(2, 2)
})
Test passed

> test_that ("expect gte", {
  expect_gte(2, 1)
})
Test passed

> test_that ("expect gte", {
  expect_gte(1, 2)
})
— Failure (Line 2): expect gte —————

1 is not more than 2. Difference: -1
Error: Test failed

```

The `expect_silent` function can be used to assert if a code produces any output or not. For example:

```

> test_that ("expect silent", {
  expect_silent(quote)
})

```

Test passed

```

> test_that ("expect silent", {
  expect_silent(str(quote))
})
— Failure (Line 2): expect silent —————

`str(quote)` produced output.
Error: Test failed

```

The `expect_true` and `expect_false` checks can be used if none of the above validations are suitable for your requirements.

```

> test_that ("expect true", {
  expect_true(TRUE)
})
Test passed

> test_that ("expect true", {
  expect_true(FALSE)
})
— Failure (Line 2): expect true —————

FALSE is not TRUE
`actual`:  FALSE
`expected`: TRUE
Error: Test failed

> test_that ("expect false", {
  expect_false(TRUE)
})
— Failure (Line 2): expect false —————

TRUE is not FALSE
`actual`:  TRUE
`expected`: FALSE
Error: Test failed

> test_that ("expect false", {
  expect_false(FALSE)
})
Test passed

```

At times, when you want to omit a test from execution, you can use the `skip` function, which takes an argument message. A meaningful text can be used for the message to be printed during execution of the tests to indicate the reason for skipping the test.

The `R CMD check` command can again be used to run the testthat tests for a package. Consider the high dimensional numerical and symbolic *calculus* package that provides high-order derivatives, ordinary differential equations, tensor calculus, Einstein summing convention, Taylor series expansion and numerical integration. The output of the check command that runs the tests is shown below: